Aya Burpee, Brinda Moudgalya
2462847, 2337294
EE/CSE 371
November 2, 2025
Lab 4 Report

Design Procedure

In this lab, we worked through two tasks relating to ASMD charts. For the first task, we worked through an algorithm that counts the number of 1s in a user-inputted binary sequence (SW[7:0] in LabsLand) and returns the result. For the second task, we built a binary search algorithm that searches through a sorted, unsigned (integer) array to find a user-inputted value (specified by SW[7:0] in LabsLand). For both tasks, KEY[3] functions as the START signal and KEY[0] functions as the RESET signal.

Task #1:

In task #1, we started by finalizing the ASMD chart that was provided to us in the lab specs. We ensured that our version of the ASMD chart was more cohesive with the style we learned about in lecture. Our version of the ASMD chart, complete with the datapath, is given below in Figure 1a:

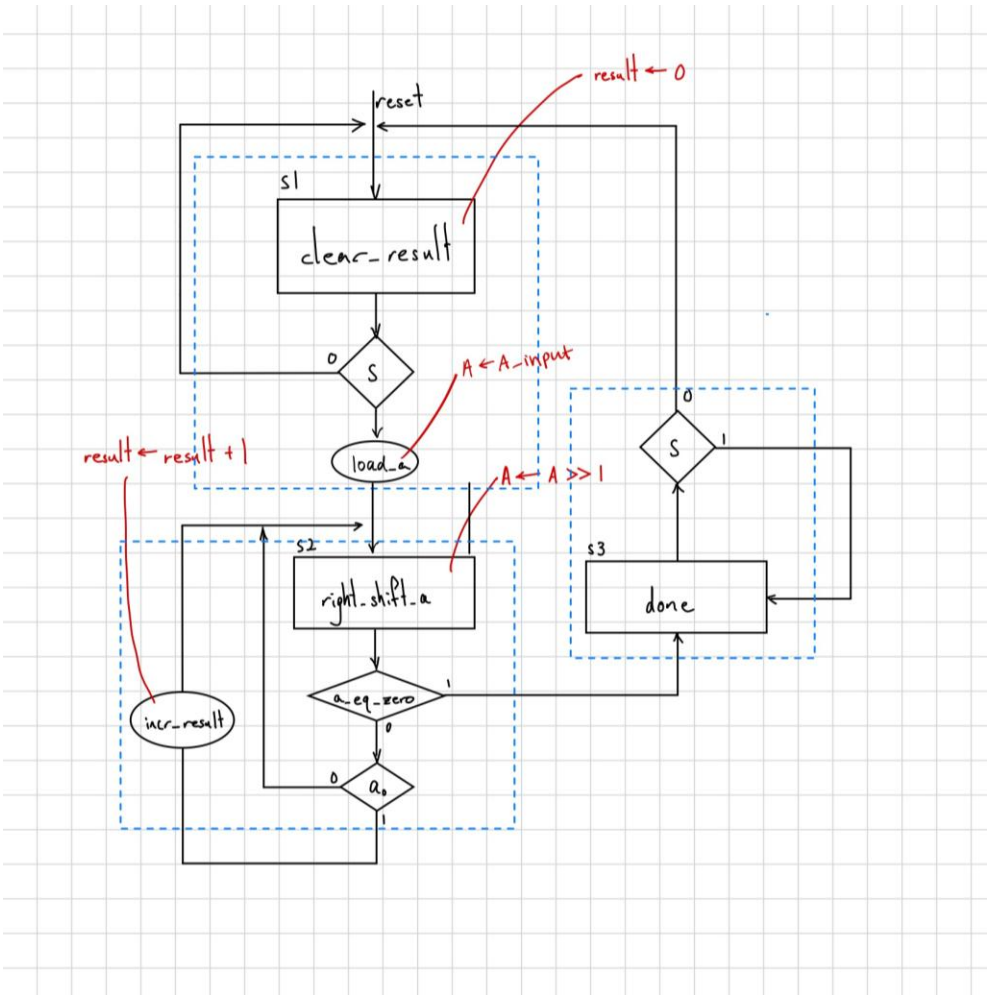


Figure 1a

To implement this diagram in SystemVerilog, we began by taking note of the lab specifications – specifically that we would be using a 50MHz clock signal, that we would be handling metastability (likely of the KEY presses), that SW[7:0] (user input), KEY0 (synchronous reset), and KEY3 (start signal) would be our inputs, and that HEX0 and LEDR9 would be our outputs to display the total number of 1s counted and the DONE signal, respectively.

To address metastability, we created a module to debounce KEY presses. This module passes inputs through two flip-flops to ensure that changing the input too close to the rising/falling clock edge would not cause metastability. Passing the inputs through these two flipflops does result in a delay of 2 clock cycles, but it also ensures no input bouncing.

Next, we implemented the actual counting functionality of the algorithm. We created two separate modules for the control circuit and the registers (datapath). The logic follows the ASMD chart in Figure 1a and looks only at the least significant bit of the input (A) to check

whether it is a 1. If it is, it increments the counter. Then, regardless of what the value of the LSB was, it performs a right shift, putting a new value into the LSB spot for the process to repeat again.

As the bit counter is running, so is the module for displaying the count on the HEX display. As in previous implementations of the display module, this utilizes a case statement to display a certain value on the HEX based on what the value of the counter is at that clock cycle.

Designer decisions:

When looking at the provided ASMD chart, we noticed a flaw – if S is equal to 1 from the beginning, then LOAD_A is never asserted, which therefore leads to unexpected behavior since A is never actually loaded in. However, as designers, we want to implement a solution that follows client specifications but also takes into account that the client-provided specs may not be the most accurate in terms of functionality. Thus, we decided not to simply implement the design given in the diagram, but to ensure that A is *always* loaded in. Thus, instead of loading if s = 0 in state one, we load if s = 1 in state 1. This way, regardless of what whether or not S is asserted from the beginning of the algorithm A is always loaded.

Task #2:

Our second task was to implement a binary searching algorithm in SystemVerilog. The algorithm searches through a pre-sorted unsigned array to try and find the user-specified integer A (which is defined by the value of SW[7:0] in LabsLand). If A exists, then the system displays A's location (its address in memory) on HEX1 and HEX0, and the FOUND signal (LEDR0) is HIGH. Once the binary searching is complete, the DONE signal (LEDR9) is pulled HIGH. The START signal is KEY3, and RESET is KEY0.

Designer decisions:

We decided not to loop back to reset after the DONE state (after the value A had been found) even if START was pulled low. This is because we wanted to keep displaying the address location of A. The behavior we implemented here for task2 is different than task1's end / DONE behavior (in task1, the display goes dark as soon as the start button is released). We implemented this behavior because when A is found through the binary searching algorithm, if START is still HIGH, it will just initialize immediately, and if A is still the same as before, then the location in memory is also the same as before.

When initializing the system, we also don't need to explicitly set FOUND to 0. This is because, upon reset, FOUND will no longer be high due to how it is assigned in the datapath – in the IDLE state, FOUND cannot be high.

We could have made the choice to store A in a separate register in the datapath to be able to store the A value when initialize is asserted, such that the value of A we're comparing the RAM values to cannot change until reset. But we chose to directly compare the RAM values to the real-time A input, which means that if the user changes the A value after FOUND goes high and the machine is in the DONE state, then although DONE still stays asserted, FOUND is no longer true (since one of the conditions for done is that data_out == A, and A has changed) and therefore the 7-segment displays go dark since they are driven by the FOUND signal. We chose this implementation because we decided it wouldn't make sense to keep displaying an address at which a previous input was found, when the current input is different. To reset the system, the user presses KEY0 again.

Taking our designer decisions into consideration, Figure 1b below shows the ASMD chart we implemented in code.

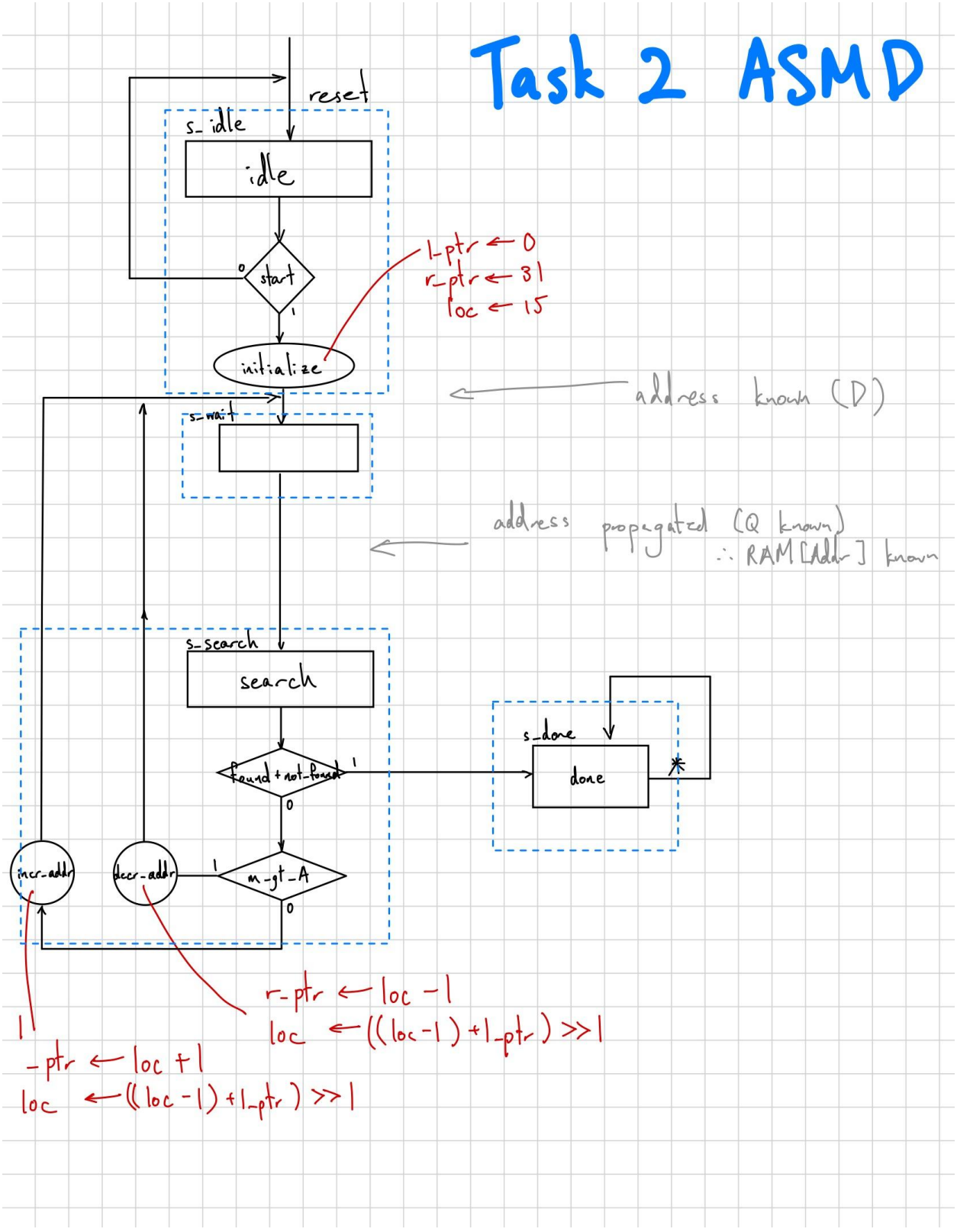

Task 2 ASMD
reset
s_idle
idle
start
0
1
initialize
l_ptr ← 0
r_ptr ← 31
loc ← 15
address known (D)
s_wait
address propagated (Q known)
∴ RAM[Addr] known
s_search
search
found + not_found
1
0
s_done
done
incr_addr
decr_addr
1
m_gt_A
0
r_ptr ← loc − 1
loc ← ((loc − 1) + l_ptr) >> 1
l_ptr ← loc + 1
loc ← ((loc − 1) + l_ptr) >> 1

Figure 1b

Our final system block diagram is given below in Figure 2, and it shows the signals and base flow of the different modules within tasks1 and 2:

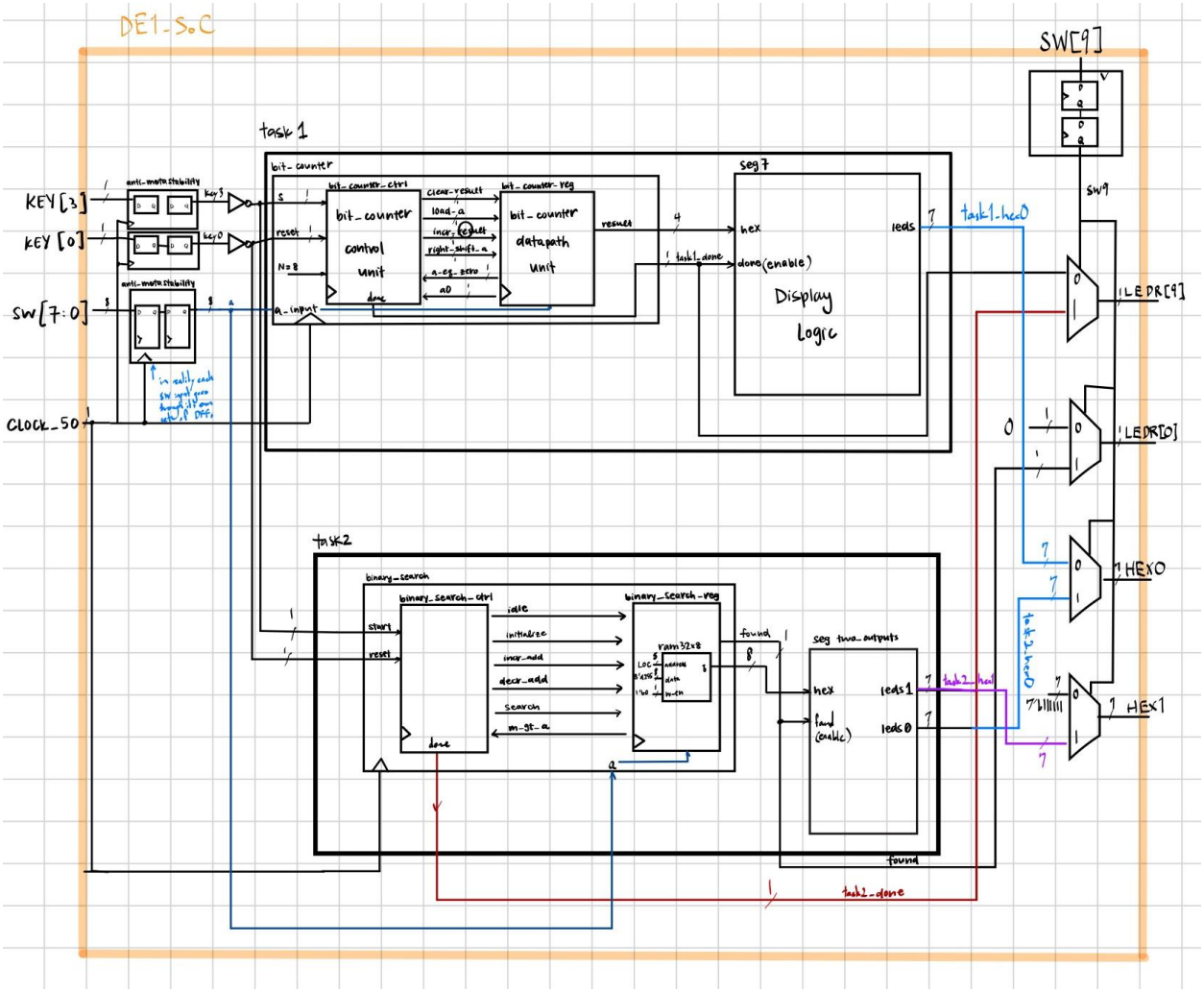


Figure 2

Results

The following section shows the results of our testbenching for tasks 1 and 2. We include the waveforms for intermediate modules, as well as task1, task2, and combined system top level testbenches. The final, combined system is able to toggle between task1 and task2 by looking at the state of SW[9]; if SW[9] is LOW, then we are implementing task1, and if SW[9] is HIGH, then we are implementing task2.

Bit counter (task1) testbench

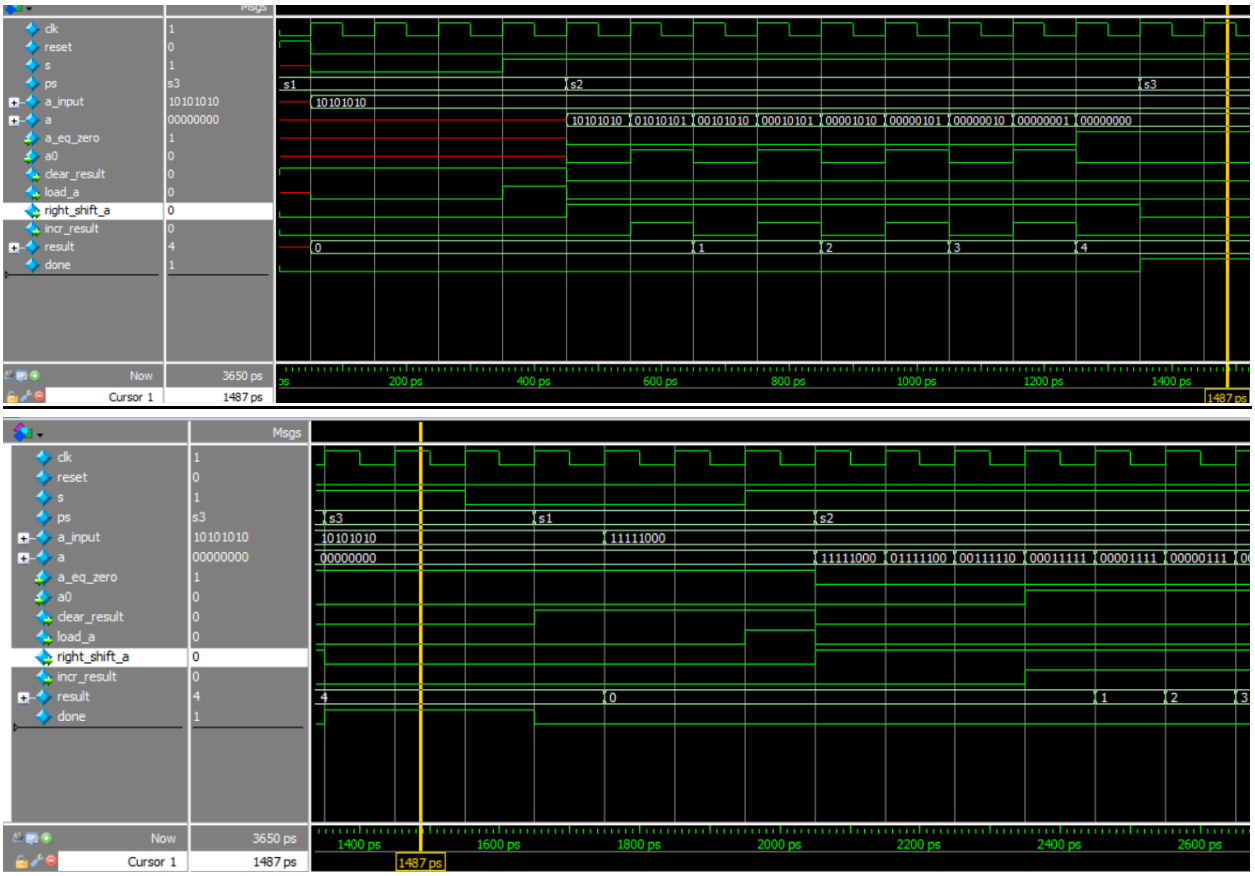


Figure 3a

The bit counter is the module that implements the counting of the 1s in the given sequence, A. From Figure 3a above, we see the user-provided value stored in *a_input*, and register *a* gets right-shifted every clock cycle. If the least significant bit of *a* is a 1, then *incr_result* is pulled HIGH and *result* is incremented by 1. Once *a* == 0, DONE is pulled high. Setting *s* to 0 will then reset the system back to state s1.

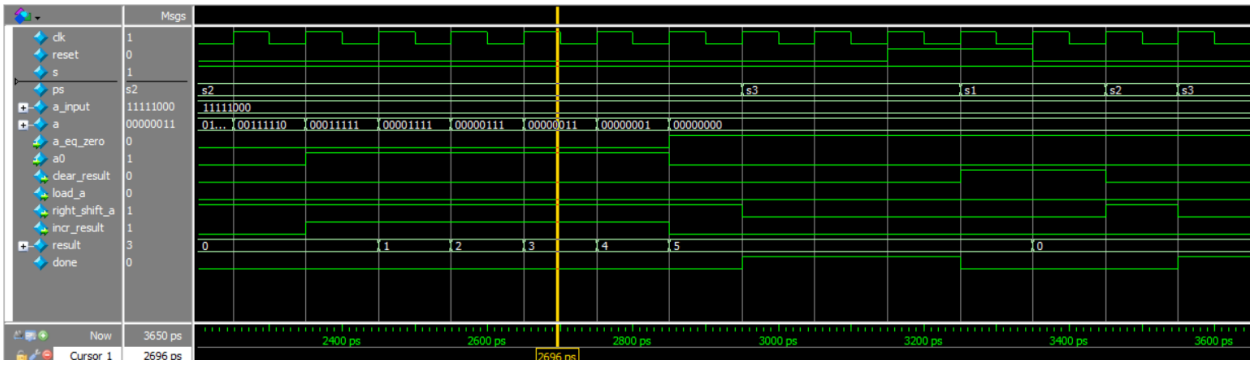


Figure 3b

In Figure 3b above, we see the results of the end of the testbench before changing the ASMD logic(to be clear, Figure 3a is from after changing the logic for load_a). Again, before changing the ASMD logic the load_a signal is asserted when in s1 and s=0, and after changing the logic, load_a is asserted when in s1 and s=1. In Figure 3b, when the system is reset (and *s* is not pulled LOW before resetting), the system resets back to state s1 and then immediately proceeds to s2 without reloading register *a_input* or *a*. This is improper behavior. After seeing this testbench, we realized we needed to modify the client specifications to ensure register *a* is always loaded before moving from state s1 to s2. Thus, we load register A whenever the present state is s1 and *s* is FALSE. By updating the logic, we get the waveforms shown in Figure 3c below, which is what we want to see – the value of *a_input* and *a* is now loaded in whenever we are at state s1, even if the *s* signal is always HIGH.

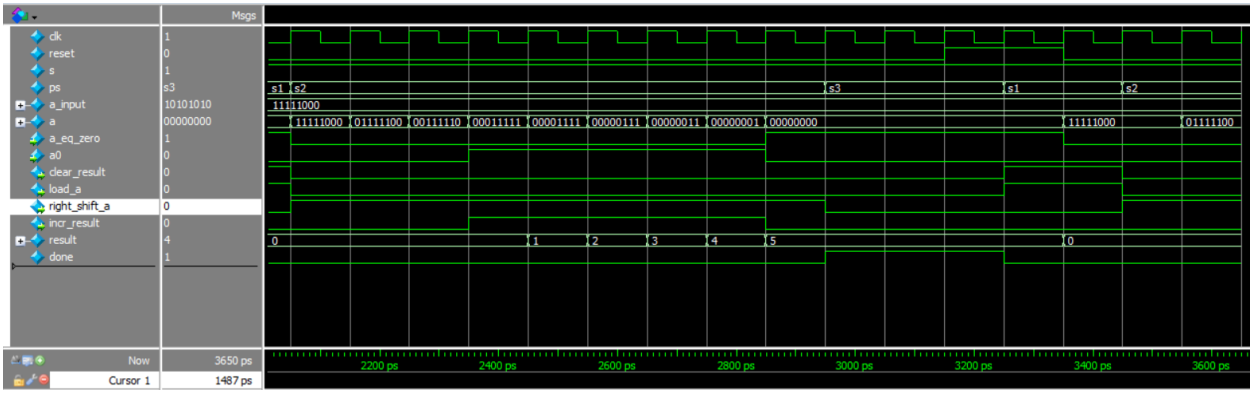


Figure 3c

To draw attention to the difference between Figure 3b and Figure 3c testbenches: we are looking at the behavior of the system after we are in state s3, and register *a* is equal to 0. If we reset without pulling *s* (the start signal) LOW, after changing the provided ASMD logic, we see that register *a* is reloaded with the value of *a_input*. In this testbench, we intentionally did not change the value stored in *a_input*, but if we had, then the new value of register *a* would follow whatever the value of *a_input* was when the system is reset.

Task1 top level testbench:

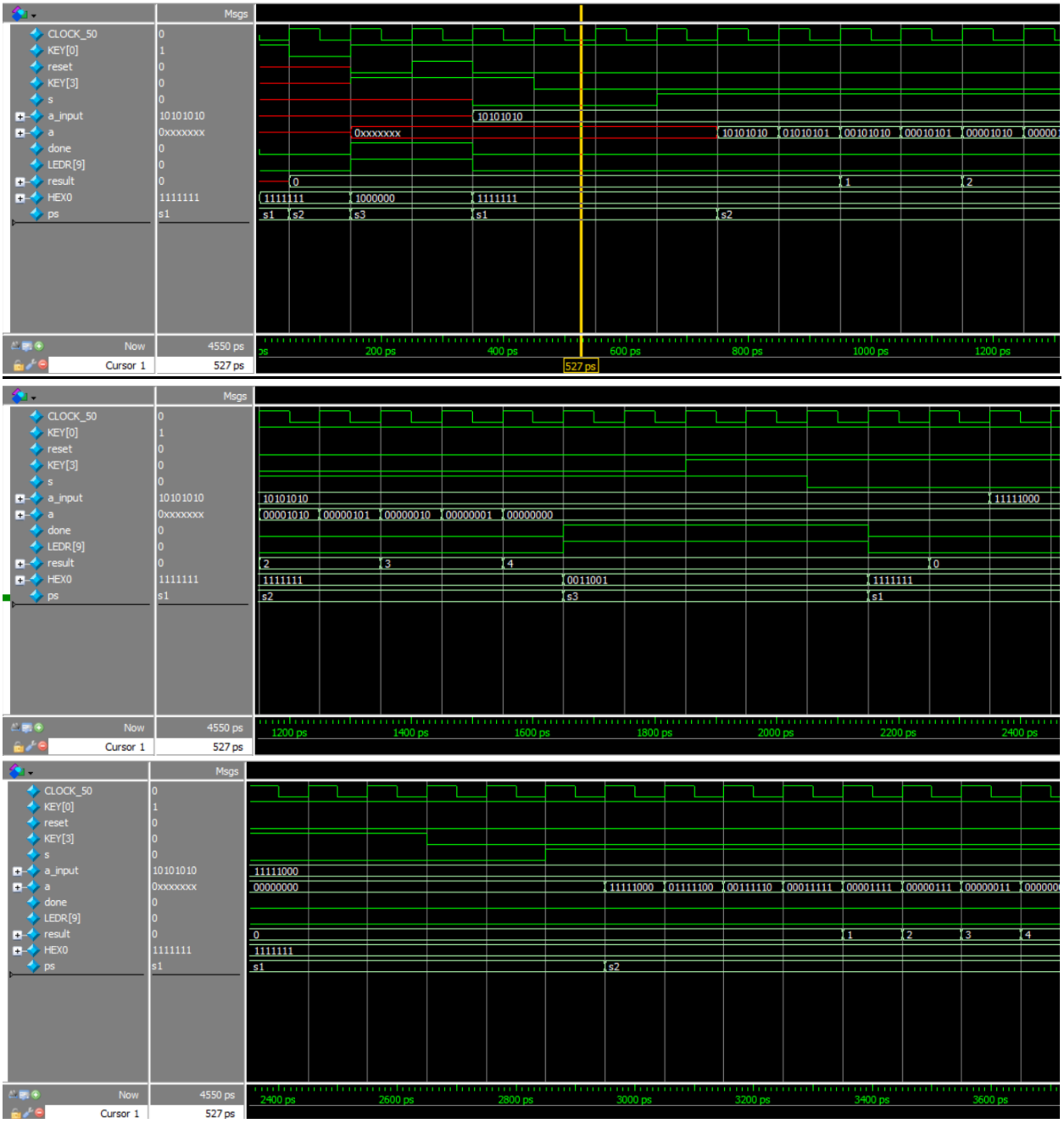


Figure 4a

Figure 4a shows the waveforms for the top-level testbench for task1. The top-level implements both the bit counter and the HEX display functionality. The HEX display functions the exact same as in previous labs, therefore we did not create a separate testbench for the HEX display and instead just used the top-level to ensure the HEX display worked as intended. For task1, the HEX display is cleared (7'b1111111) until DONE is pulled HIGH, after which it displays the value of

RESULT which is the result of the bit counter module. This behavior is seen in the above Figure 4a.

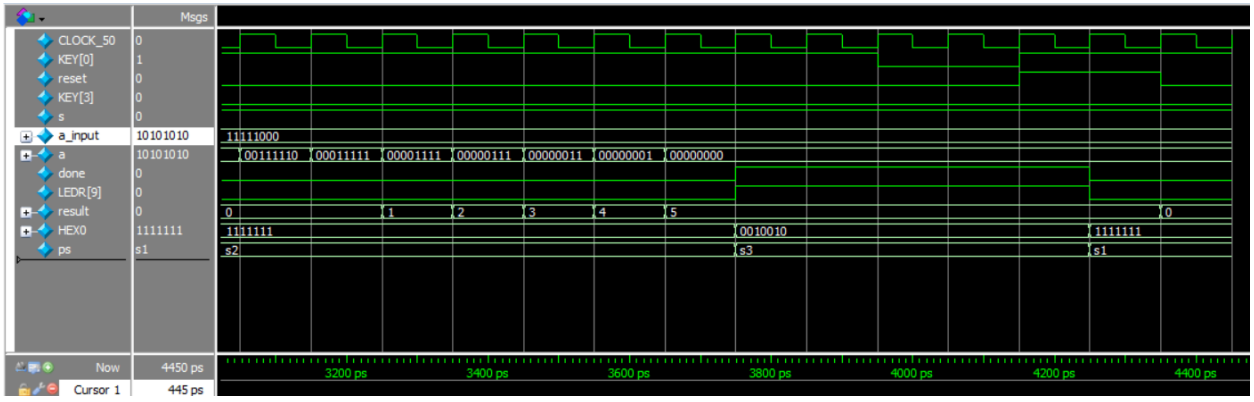


Figure 4b

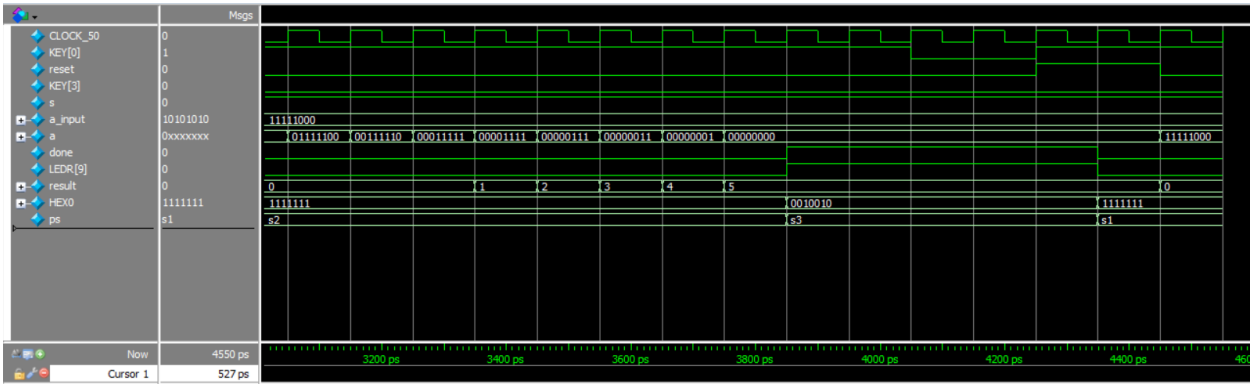


Figure 4c

Figure 4b and Figure 4c show the difference between the testbenches of the client-specified ASMD chart and the fixed implementation where we ensured register *a* is always loaded before moving from state s1 to s2. As we can see, Figure 4c shows the proper implementation, where resetting without first pulling *s* low still reloads the value of *a_input* into register *a* instead of keeping it at 0. (note that there is a 1 clock cycle shift in the figures because we added an extra clock cycle in the beginning when testbenching in Figure 4c)

Binary search (task2) testbench:

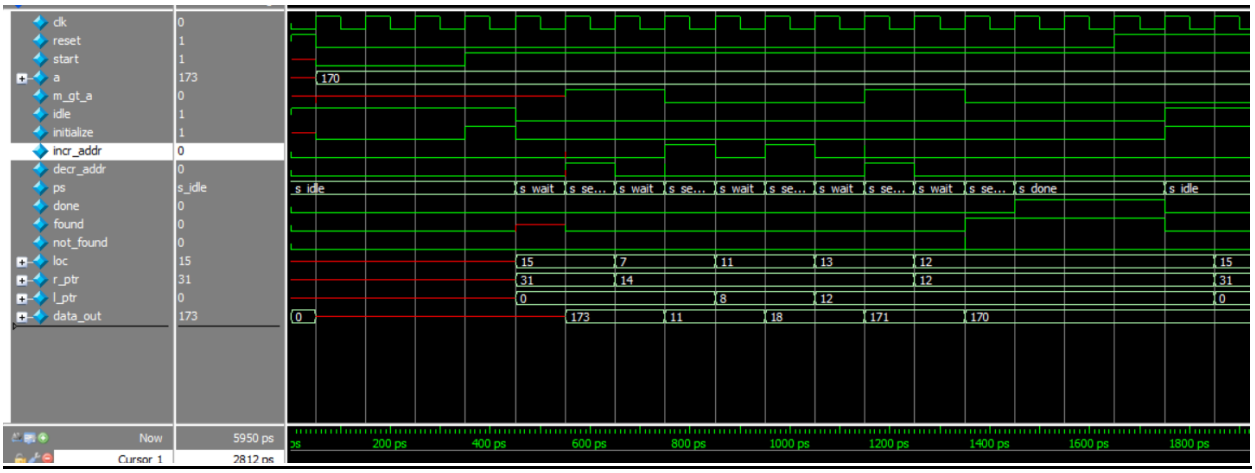


Figure 5a

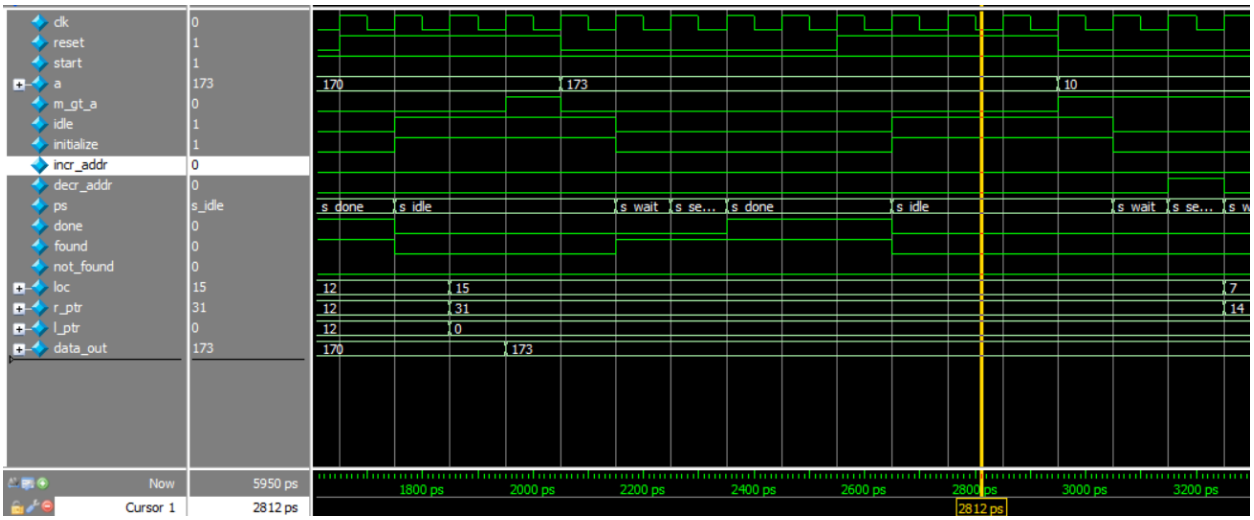


Figure 5b

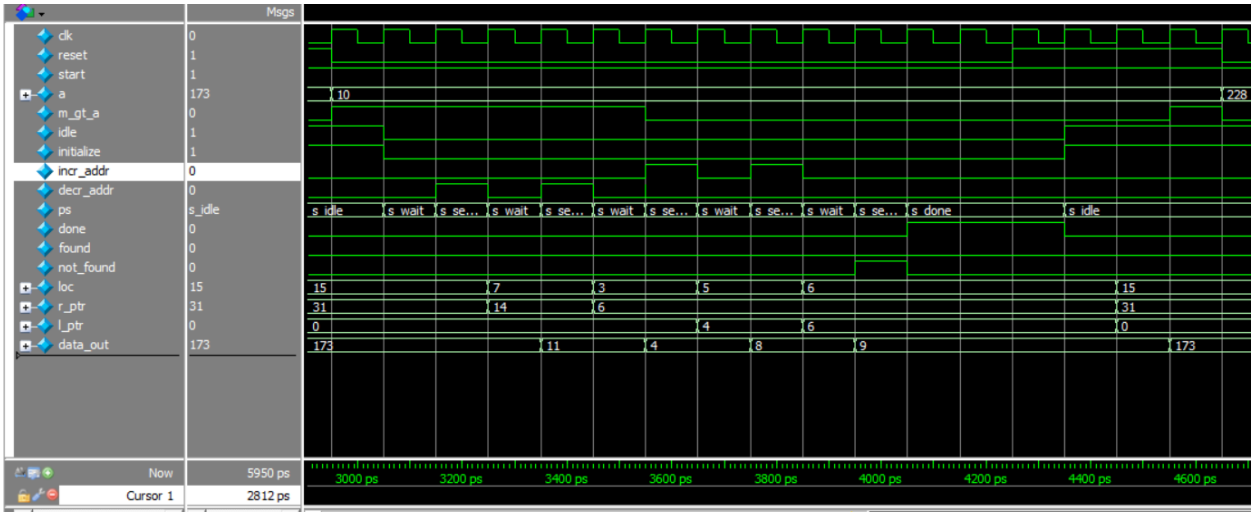


Figure 5c

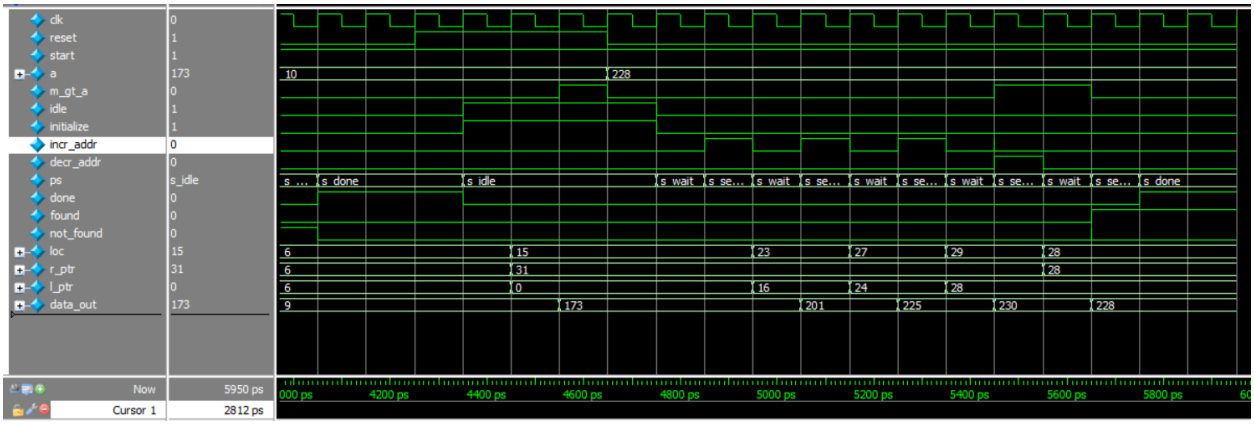
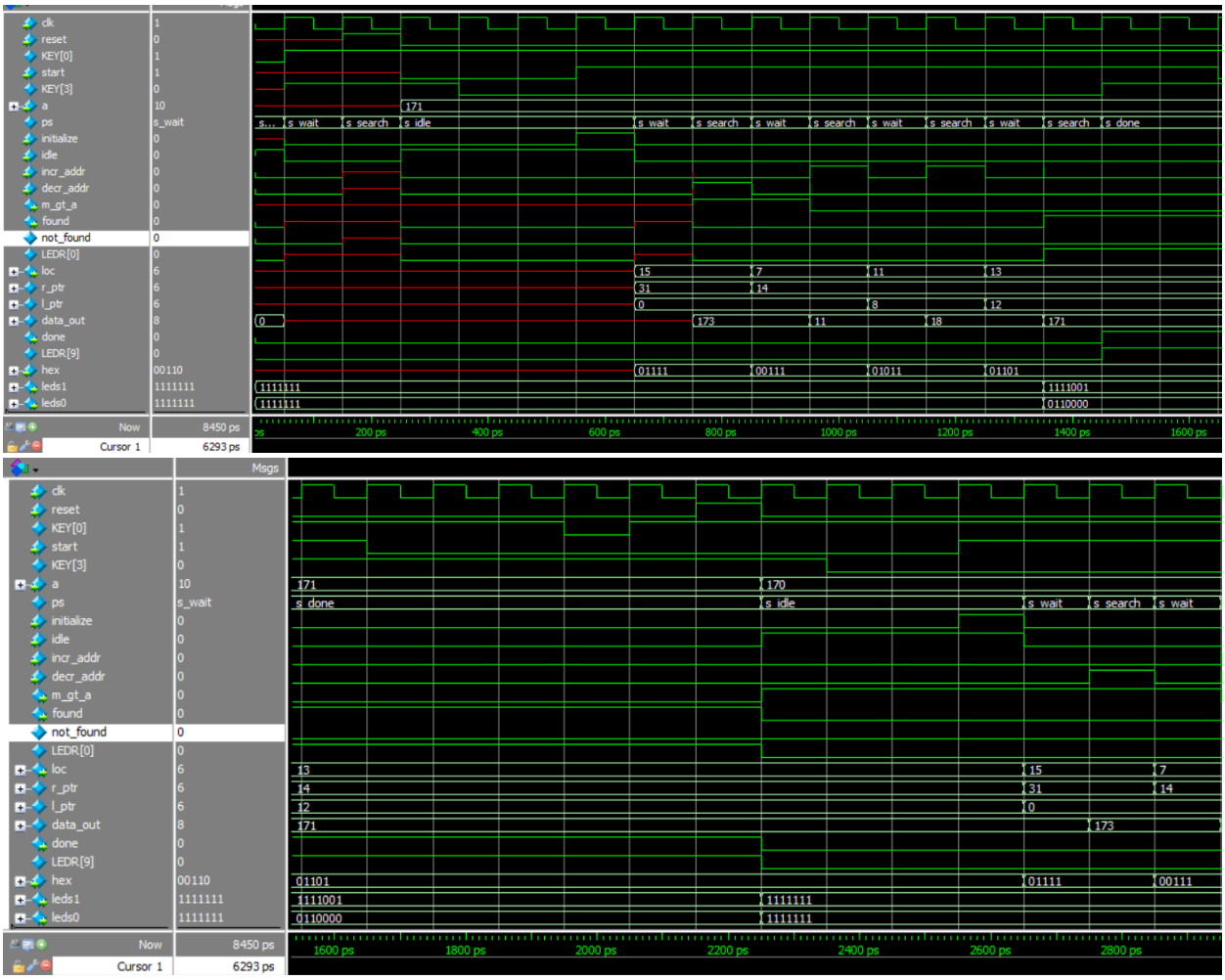


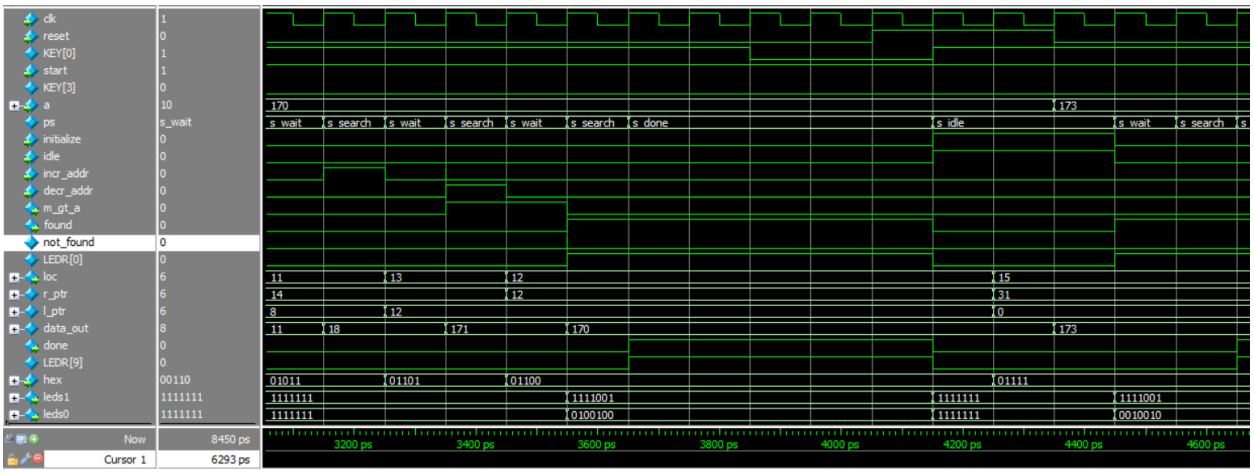
Figure 5d

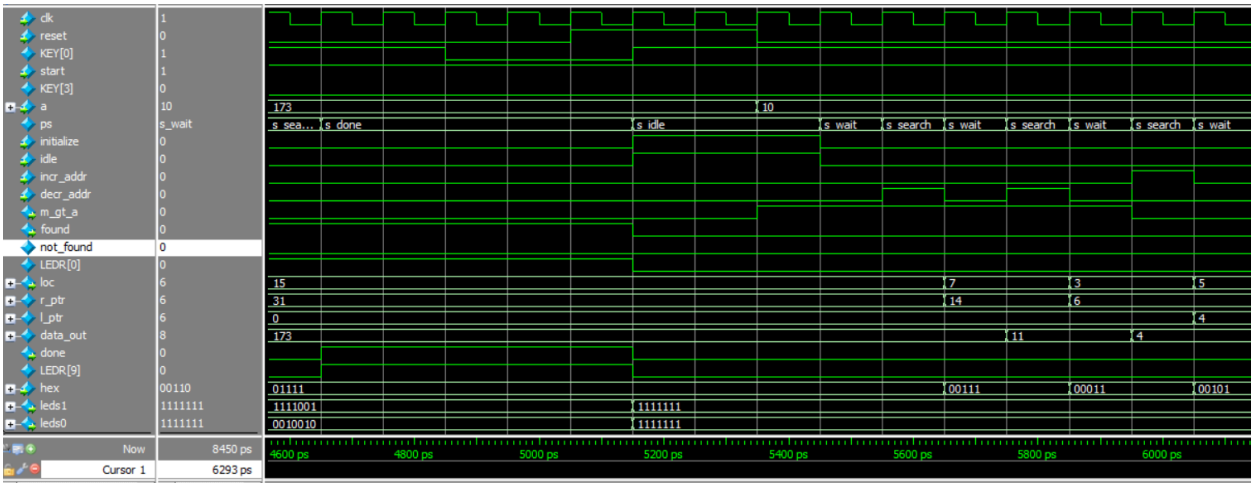
Figures 5a through d shows the working of the binary search module. This is the main functionality of task2, which is to search through a sorted, unsigned array for a user-specified value (which we call *a*). We use a right- and left-pointer as the highest and lowest indices of the array, and a middle pointer called Loc (short for “location”). We compare the value stored at Loc to *a*; based on whether the value stored at Loc is greater than *a*, we update the value of Loc and the right and left pointers accordingly. When the value stored at Loc is equal to *a*, we return Loc, pull DONE high, and pull FOUND high as well. If, after performing binary search down to the last address, we cannot find *a* in the sorted array, then we still pull DONE high (driven by a not_found signal), and we pull NOT_FOUND high (keeping FOUND low). This behavior is seen in Figure 5c: the value 10 is not stored in the array, but we can see that after not_found goes high, the present state goes to the done state.

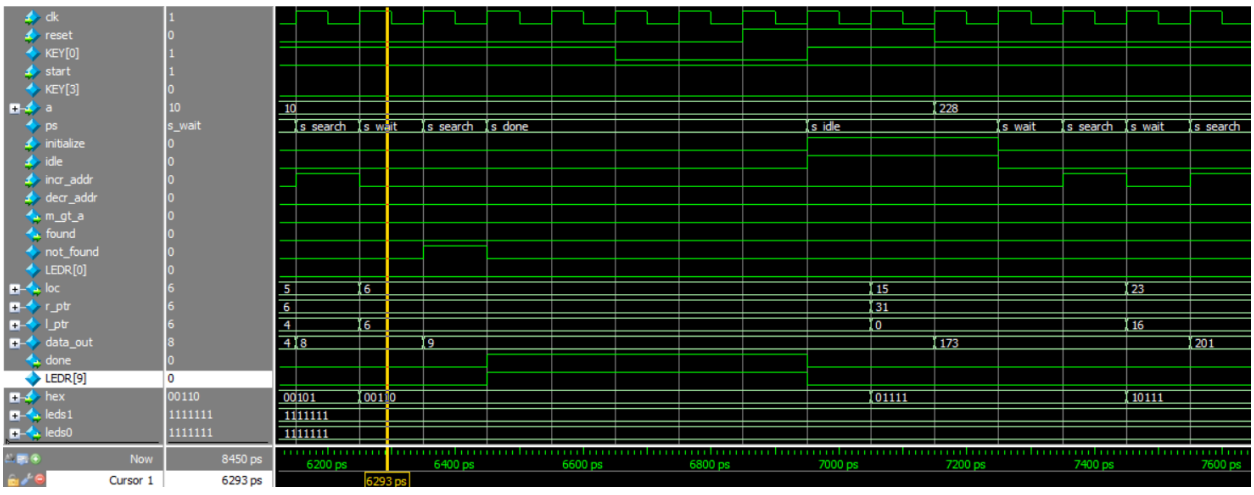
Task2 top level testbench

KEY[0] (reset) and KEY[3] (start) are passed through the anti-metastability module, which results in their values being changed with a two-clock cycle delay. LEDR[0] and LEDR[9], however, are pulled HIGH as soon as the FOUND and DONE signals are. The HEX displays show the target address as soon as FOUND is asserted. This was a design choice, where we wanted the visual display to update as soon as the actual internal signals did, instead of after a delay.




clk 1
reset 0
KEY[0] 1
start 1
KEY[3] 0
a 10
ps s_wait
initialize 0
idle 0
incr_addr 0
decr_addr 0
m_gt_a 0
found 0
not_found 0
LEDR[0] 0
loc 6
r_ptr 6
l_ptr 6
data_out 8
done 0
LEDR[9] 0
hex 00110
leds1 1111111
leds0 1111111
Now 8450 ps
Cursor 1 6293 ps
s_wait s_search s_wait s_search s_wait s_search s_done s_idle s_wait s_search
3200 ps 3400 ps 3600 ps 3800 ps 4000 ps 4200 ps 4400 ps 4600 ps


clk 1
reset 0
KEY[0] 1
start 1
KEY[3] 0
a 10
ps s_wait
initialize 0
idle 0
incr_addr 0
decr_addr 0
m_gt_a 0
found 0
not_found 0
LEDR[0] 0
loc 6
r_ptr 6
l_ptr 6
data_out 8
done 0
LEDR[9] 0
hex 00110
leds1 1111111
leds0 1111111
Now 8450 ps
Cursor 1 6293 ps
s_sea... s_done s_idle s_wait s_search s_wait s_search s_wait s_search s_wait
4600 ps 4800 ps 5000 ps 5200 ps 5400 ps 5600 ps 5800 ps 6000 ps


clk 1
reset 0
KEY[0] 1
start 1
KEY[3] 0
a 10
ps s_wait
initialize 0
idle 0
incr_addr 0
decr_addr 0
m_gt_a 0
found 0
not_found 0
LEDR[0] 0
loc 6
r_ptr 6
l_ptr 6
data_out 8
done 0
LEDR[9] 0
hex 00110
leds1 1111111
leds0 1111111
Now 8450 ps
Cursor 1 6293 ps
s_search s_wait s_search s_done s_idle s_wait s_search s_wait s_search
6200 ps 6400 ps 6600 ps 6800 ps 7000 ps 7200 ps 7400 ps 7600 ps
6293 ps

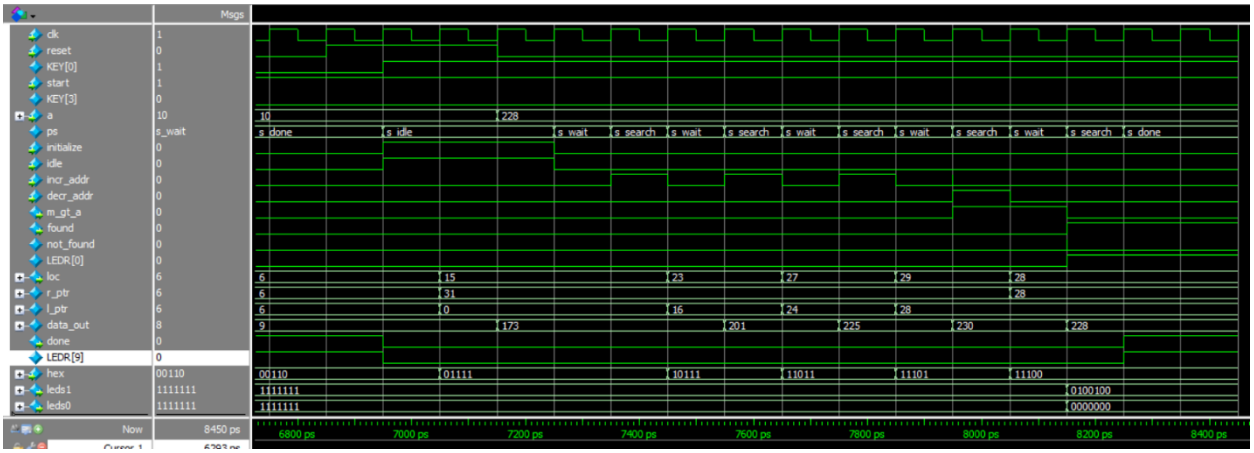


Figure 6

Figure 6 shows the testbench for the entirety of task2. This includes the binary search algorithm from Figure 5, as well as the implementation of the HEX display. The HEX display functions the same as task1, except that again, it is only displayed when FOUND is HIGH (in task1, the HEX display was driven by DONE).

Going through the waveforms, we see that upon resetting, we load a value (taken from the status of SW[7:0]) into register *a*. We then initialize the left and right pointers to either end of the sorted array and set Loc to the middle of the two pointers. Then, we repeatedly compare the value of *data_out* (the value at the current index (Loc) of memory) to the value we want (*a*). If the value of *data*_out is equal to a, then we have found the value; FOUND is pulled HIGH, DONE is pulled HIGH, and the value of Loc is returned (and displayed on the HEX display). If the value of *data_out* is greater than *a*, then we move the right pointer to one index below Loc. If, instead, the value of *data_out* is less than *a*, then we move the left pointer to one index above Loc. After this operation, we take the average of the values of the right and left pointers and set Loc to that index.

This is the general working of the binary searching system. If, at the very end (when the right and left pointers are right next to each other), *a* has not been found in the sorted array, then DONE is still pulled high but there is no display on the HEX—we can see then in Figure 6 when we set our target value to 10, which doesn't exist in the array.

Overall combined top level testbench (DE1_SoC)

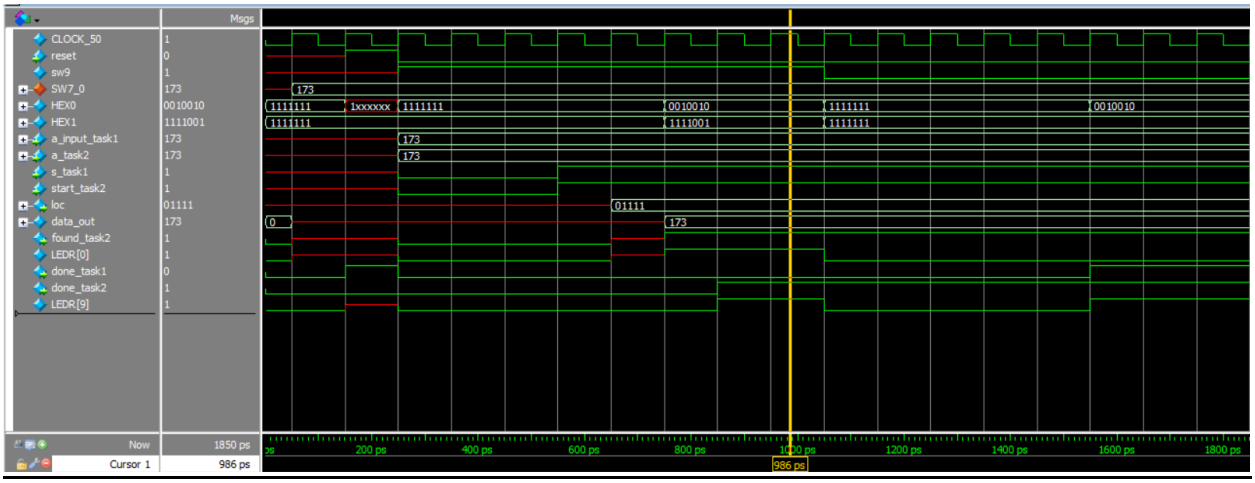


Figure 7

The full-system top level module essentially just integrates both task1 and task2 into one system. The toggling between tasks is handled by SW[9]; when SW[9] is LOW, task1 is running, and when SW[9] is HIGH, task2 is running.

Input *a* is the same for both tasks, since we are feeding the same signal into both modules. This is also true of the *s*/*start* signals.

In the waveforms shown in Figure 7, SW[9] starts off HIGH which means we are implementing task2. It searches through the array for the number 173, and when it is done it asserts DONE as well as FOUND. The HEX display is updated accordingly, to 15 (which is the address that 173 is found at).

When we switch SW[9] to LOW, since task1_done is still low (only task2_done is high), the HEX display turns off. This is expected behavior so the output of task1 does not depend on the output of task2.

We then see that when SW[9] is LOW, the system counts the number of 1s in the binary sequence for the decimal number 173. When the DONE signal is asserted, a “5” is returned in HEX, showing that there are five 1s in the binary sequence. 173 in binary is 10101101. This does indeed contain five 1s.

For task2, we deliberately chose the value stored in the middle address (Address 15 stores 173), which is the address that’s checked first for the target value, so that by the time task2 finishes, task1 is still not done. This way, we could check that the outputs properly switch over with SW[9], since if we switch over right after task2 is complete, the hex displays and LEDR for task1 should go off, and back on when task1 is finally complete.

We also see that once the done_task1 signal goes high, LEDR[9] goes high, and HEX0 turns on. HEX1 stays off because task1 only uses HEX0.

Flow Summary:

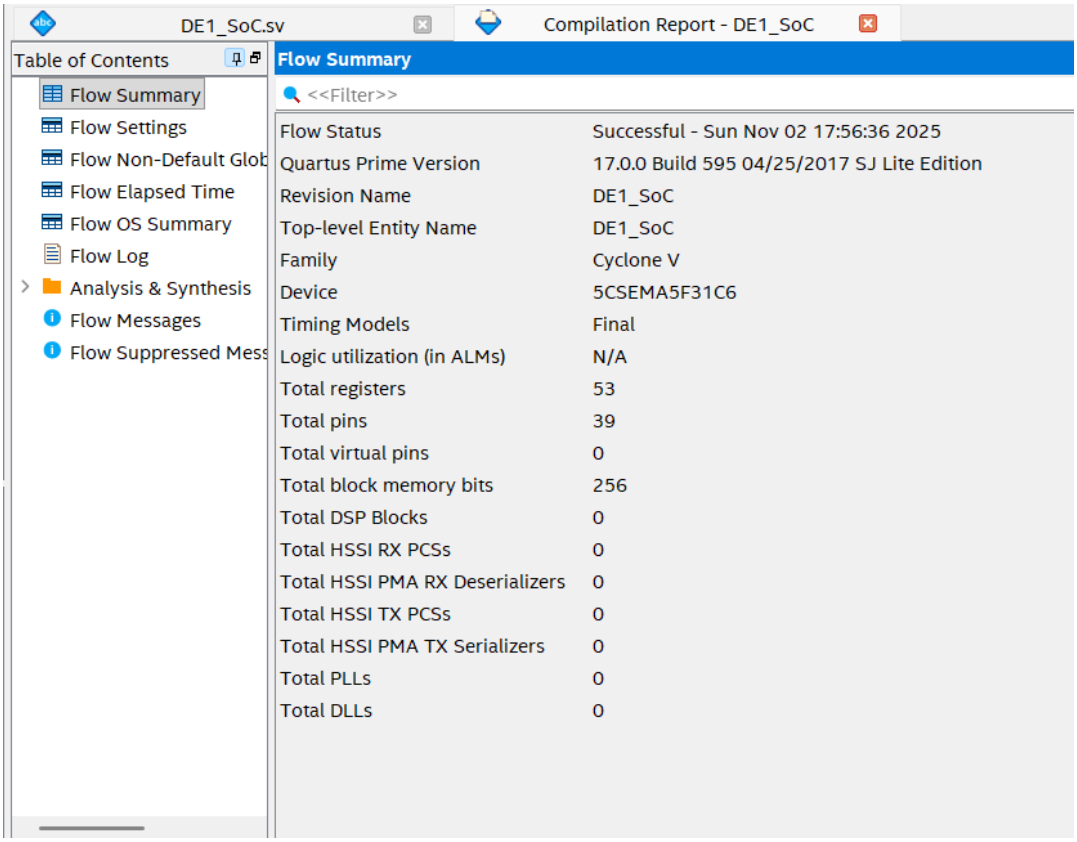


Figure 8

Experience Report:

Task1 was nice to implement, and task2 felt like it was straight from a computer science class which was not as enjoyable. There were no significant issues when completing this lab.

Estimated total time: 30 hours

Reading lab document: 0.5 hours

Planning: 1.5 hours

Designing: 2.5 hour

Coding: 5 hours

Testbenching: 2 hours

Redesigning: 1.5 hours

Recoding: 4 hours

Re-testbenching: 2 hour

Redesigning and recoding: 4 hours

Re-testbenching: 2 hour

Documentation: 5 hours